

AI-Powered IPL Auction Simulator with Impact Parameters — Full Implementation Plan (v3, final)

Confirmed scope:

- Base player pool: **IPL 2025 mega-auction list**
 - **Retention is a gameplay phase, not history** — each team (human or AI) picks up to 6 retentions from their **actual 2024 squad**, under real 2025 slab rules, before the auction begins
 - **10 franchises** in play; any team without a human assigned is AI-controlled for both retention and bidding
 - Purse rules: **real IPL 2025 numbers** (₹120cr, real slabs) — using invented numbers would break the retention-cost logic anyway, so this was really settled by the retention decision
 - Impact Parameters: acquisition-stage only, feeding the bidder + a live "suggested value" badge
-

1. Impact scoring — final decision: calibrated formula, not a black box

You asked for "best results" — here's the reasoning for the approach:

A **pure hand-tuned formula** (fixed weights like "30% average, 25% strike rate...") is explainable but the weights are guesses. A **full ML model** would give no explanation and needs training data we don't have yet.

The better middle path: keep the formula *structure* transparent (same explainable components: batting/bowling sub-metrics, experience, recency), but **calibrate the weights empirically** against real historical auction outcomes — IPL 2023/2024/2025 actual sold prices are public data. We regress each player's stat-line against what the market actually paid for comparable players, freeze the resulting coefficients into a static config, and ship that. This gives:

- Real accuracy (grounded in what actually happened at real auctions, not guesswork)
- Full explainability (it's still a visible weighted formula — the weights just came from data instead of intuition)
- No live black-box risk, no training infra needed at runtime

This is a one-time offline calibration step (`scripts/calibrate_impact_weights.py`), re-run manually if you want to refresh it with newer auction data. The **v4 upgrade path** (noted, not built now): once enough *simulated* auctions have run in-app (logged via the Mongo replay store), retrain against your own game's outcomes too, blending real-world calibration with in-app behavior.

2. Retention phase — new core feature

This sits **before** the main auction, as its own state in the room lifecycle:

`LOBBY → RETENTION → IN_PROGRESS (auction) → COMPLETED`

Data needed: each franchise's final 2024 squad (player list) — sourced from Wikipedia's 2024 IPL season squad pages (stable infobox/squad-table format, free, reliable).

Master player pool construction (important nuance): since retention choices are now player-driven rather than historical, we need a merged universe: `master_pool = (2024 squads, all 10 teams) ∪ (2025 mega-auction list, 574 players)` This is necessary because some players who were *actually* retained in real life never appear in the 574-player 2025 auction list (they'd never have been auctioned) — but in our version, a team might choose *not* to retain them, so they must be able to fall into the open pool. Conversely, someone in the 2025 auction list might get retained here instead. Implementation detail: players from the 2024 squads who lack a 2025 reserve price get a **role/category-based fallback reserve price** (e.g., last known base price if available, else lowest bracket ₹30L) — flagged in the data so it's not a silent guess.

Retention flow (per team, human or AI):

1. See your 2024 squad list with each player's role + Impact Score.
2. Pick 0-6 players to retain, respecting: max 5 capped, max 2 uncapped, total ≤ 6 .
3. Real slab costs auto-deduct from the ₹120cr purse as you pick (1st capped retention – ₹18cr, 2nd – ₹14cr, 3rd – ₹11cr, 4th – ₹18cr, 5th – ₹14cr; uncapped flat – ₹4cr each), so teams see live remaining purse as they decide.
4. On confirm: `RTM cards granted = 6 - total retained`.
5. AI teams run an `AIRetentionAgent` (extends the same rule-based bidder logic): retains the highest-Impact-Score-relative-to-cost players first, biased toward filling role gaps, and deliberately leaves some purse/RTM flexibility rather than always maxing out 6 (mirrors real franchise behavior — not every real team retained 6).
6. Once all 10 teams finish (or timeout → AI auto-completes for slow/absent humans), engine computes the **final open auction pool** = `master_pool - all retained players`, and room transitions to `IN_PROGRESS`.

RTM usage in the live auction: when a non-owning team wins a bid on a player, if his 2024 franchise still holds an RTM card, that franchise gets prompted to exercise it; if used, the original highest bidder gets one final chance to raise, then the RTM team locks the player at that final figure (mirrors the real 2025 rule exactly) — implemented as an extra optional step in the lot-closing logic, not a new state machine.

3. Updated data model additions

squad_2024_entries (id, franchise_team, player_id (FK), role, was_capped)

retention_slabs (static config table, not user-editable at runtime) (slot_no (1-5), capped_cost_cr, uncapped_cost_cr (flat 4 regardless of slot))

team_retentions (id, room_id, team_id, player_id, retention_cost_cr, slot_no, is_uncapped)

rtm_cards (id, room_id, team_id, cards_remaining)

player_stats, **venues** — unchanged from v2 plan.

teams.purse_remaining_lakh now initializes as **12000** (₹120cr in lakh) - **sum(retention costs)** rather than a flat starting number.

4. Updated repository structure (additions only)

```
backend/
├─ app/
│   └─ engine/
│       └─ retention_engine.py      # new: retention state machine +
validation
│       └─ rtm_handler.py          # RTM prompt/resolve logic during lot
closing
│       └─ ai_service/
│           └─ ai_retention_agent.py  # new: AI retention picks
│           └─ impact_engine.py      # now loads calibrated weights from
config
├─ scripts/
│   └─ scrape_squad_2024.py        # Wikipedia 2024 squad pages →
squads.json
│   └─ calibrate_impact_weights.py  # regress stats vs historical sold
prices → weights.json
│   └─ build_master_pool.py         # merges 2024 squads + 2025 auction
list, dedupes, fills fallback reserve prices
frontend/
├─ src/pages/
│   └─ RetentionPhase.tsx          # new: per-team retention picker UI,
live purse/slab display
├─ src/components/
│   └─ RTMPrompt.tsx              # new: "Use your RTM card?" modal
during auction
```

5. Build phases (final)

Phase	Deliverable
0	Repo scaffold: backend venv + FastAPI, frontend Vite+Tailwind, Postgres connection, health check.
1	Player-pool ETL: BCCI 2025 auction-list PDF → <code>players</code> ; Wikipedia 2024 squads → <code>squad_2024_entries</code> ; <code>build_master_pool.py</code> merge + fallback pricing.
2	Stats ETL: Statsguru export + Cricsheet aggregation → <code>player_stats</code> ; Wikipedia venues → <code>venues</code> .
3	<code>calibrate_impact_weights.py</code> against 2023-2025 historical sold prices → frozen <code>weights.json</code> ; Impact Engine v1 built on it, sanity-tested against known players.
4	Room/team CRUD (REST): create/join room, assign up to 10 franchises to humans, rest flagged AI.
5	Retention engine + UI: per-team retention picking, slab deduction, RTM card allocation; AI retention agent.
6	Core auction engine (WS): lot flow, bid validation, timer, sold/unsold; RTM prompt/resolve on lot close.
7	Contextual Impact Scores wired into lot broadcast; AI bidder v1 (retention-aware purse state as starting point).
8	Frontend polish: Lobby → Retention → AuctionRoom → Results, Impact badges, purse/RTM indicators, charts.
9	Mongo integration: raw catalogs, audit log, full replay export (useful later for v4 ML calibration).
10	Full 10-seat LAN test (mix of human/AI), edge cases (max-6 retention limits, RTM-after-6-retentions blocked, overseas/squad caps, purse floor), then deploy.

6. Nothing else outstanding — ready for Phase 0

All prior open questions are now resolved. Next step is scaffolding: backend venv + FastAPI skeleton with Postgres wired up, and the frontend Vite+Tailwind skeleton, so Phase 0 is demoable before we touch scraping or engine logic. Confirm and I'll start writing that code.